CONFIDENTIAL — TECHNICAL INVENTION DISCLOSURE

StoriMac: A Multi-Stage AI-Assisted Narrative Development Platform

System architecture, data flow, canon-state management, AI/ML implementation, retrieval methodology, and consistency-checking mechanisms, prepared for intellectual-property assessment.

Prepared for	Business Analyst / IP counsel review
Subject system	StoriMac (multi-project fiction/screenplay development platform)
Source basis	Direct review of the production source repository (Next.js / Firebase / Anthropic Claude)
Date prepared	26 August 2026
Document status	Technical brief — factual disclosure, not legal claim language

This document describes technical mechanisms as implemented in the current source tree. Sections are marked to distinguish what is built and verified in code from what is specified only in product requirements documents but not yet implemented.

Contents

1. Field and Overview

1.1 Problem Addressed

1.2 Summary of the Approach

2. System Architecture

2.1 The Shared Canon Engine

2.2 Per-Project Engines: Configuration, Not Reimplementation

3. Data Flow — The Turn-Processing Pipeline

4. Canon Management Mechanism

4.1 The Canon State Machine

4.2 Systemic Dependency Graph

4.3 Conflict Resolution Protocol

5. AI/ML Implementation Details

5.1 Model and Forced Structured Output

5.2 Per-Project System Prompts

6. Retrieval and Grounding Methodology

6.1 Cross-Project Structured Field Grounding

6.2 Keyword Retrieval Over the Story-Format Library

6.3 What Is Explicitly Not Present (Training / Fine-Tuning / Embeddings)

7. Consistency-Checking Processes

7.1 Prompt-Level Scope Boundaries

7.2 Passive Heuristic Guardrail Logging

7.3 Deterministic Stage Audit

7.4 Causal-Chain Traceability Gate

7.5 Retrieval-Metadata Boundary Enforcement

8. Distinguishing Technical Mechanisms

9. Technical Effects Achieved

10. Current Implementation Status and Known Limitations

11. Glossary

Appendix A — Source File Inventory

1. Field and Overview

1.1 Problem Addressed

StoriMac is a web application that guides an author through a sequence of connected creative-development stages — a Story Foundation, a Character Bible, a World Bible, and (in progressively later stages) a Screenplay Architecture and scene-by-scene draft — using a conversational large-language-model (LLM) interface for each stage. Each stage consumes the structured output of the stage(s) before it as grounding input.

Systems that use a conversational LLM as the primary authoring interface for long-form, multi-session creative work face a specific technical problem: an LLM's output is probabilistic free text, and a chat transcript alone does not constitute a reliable, queryable record of which facts the author has actually decided versus which facts the model merely suggested, explored, or restated. Left unmanaged, this creates two concrete failure modes: (a) *canon drift* — the model silently contradicts or drifts away from a fact the author already confirmed, because nothing distinguishes a confirmed fact from a passing suggestion in the conversation history; and (b) *unsafe concurrency* — because a single LLM call over a full conversation window can take many seconds, a naive "read the current state, wait for the model, write the whole state back" implementation can silently discard an explicit, fast user action (e.g., a button click) that happened while the model call was still in flight.

1.2 Summary of the Approach

StoriMac addresses this by treating the LLM strictly as a text-and-suggestion generator and layering a separate, deterministic, code-level state machine — the **Canon Engine** — on top of it. Every turn of conversation is forced, at the API level, to return not only a natural-language reply but also a machine-checkable structured payload describing what the model currently believes the state of each tracked story element to be. That structured payload is independently schema-validated, passed through explicit transition rules, and persisted using a field-level (rather than document-level) write strategy that keeps the model's *proposals* and the author's *explicit decisions* in disjoint, independently-writable state, specifically to prevent the two race and drift failure modes described above. Sections 2–9 describe the concrete mechanisms that implement this approach, each grounded in source code read directly from the production repository.

2. System Architecture

The platform is organized as a single Next.js application with a single Firestore database and a single shared state-management library — the Canon Engine — that every creative-development "Project" (P1 Story Foundation, P2 Character Bible, P3 World Bible, and later stages) configures rather than reimplements.

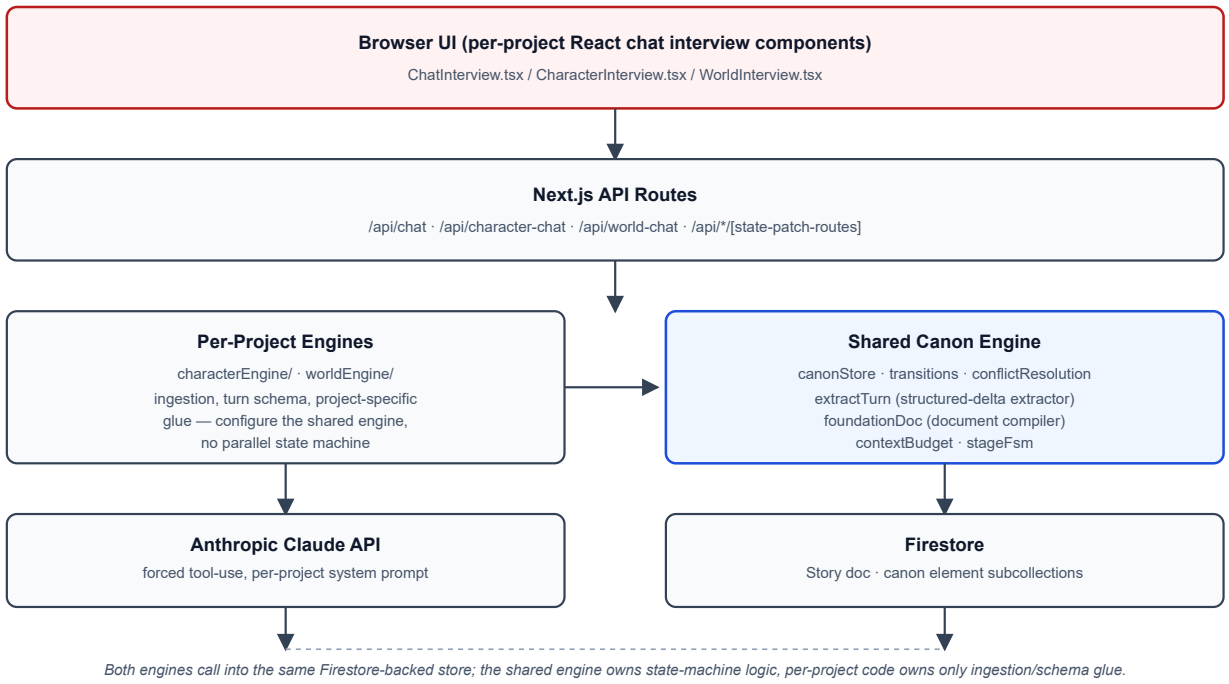


Figure 1 — High-level system architecture: shared Canon Engine consumed by per-project engines, sitting between per-project API routes and Firestore.

2.1 The Shared Canon Engine

The Canon Engine lives at `web/src/lib/canonEngine/` and is the single place where canon-state semantics are defined. Its components:

File	Function
`types.ts`	Defines the `CanonElement` record (id, status, depth mode, value, rationale, dependency list, retrieval code, history log) and the closed status enum `Exploring | Working | Confirmed | Parked`. Declared generic across every project.
`transitions.ts`	`isValidTransition()` — a lookup table enforcing which state transitions are structurally legal (e.g. a change to an already-Confirmed element cannot silently bypass the conflict-resolution path).
`canonStore.ts`	The transactional persistence layer. `applyStateDelta()` writes an entire turn's element changes inside a single Firestore transaction (atomic — no partial writes), rejects an unguarded overwrite of a Confirmed element via `CanonConflictError`, and appends an audit-trail entry on every write. Parameterized by a collection

	name, so multiple projects reuse the identical store code against different Firestore subcollections.
`stageFsm.ts / stageDefinitions.ts`	The Project-1 stage-gate state machine (an 8-stage workflow). Split into a pure-data file and a logic file specifically so the data can be safely imported into client-side code without pulling in server-only Firebase Admin bindings.
`elementRegistry.ts`	The closed vocabulary of valid element identifiers for Project 1, fed directly into the LLM tool schema's enum constraint so the model cannot invent new field names.
`stateDelta.ts / extractTurn.ts`	The structured-output contract and the generic extraction/validation routine — detailed in Section 3.
`conflictResolution.ts`	Conflict detection and resolution — detailed in Section 4.3.
`contextBudget.ts`	Bounds how much conversation history is replayed into the model call — detailed in Section 3.
`storyStore.ts`	Story/session persistence: creation, resumption, message logs, and every project's own state-slice read/write functions.
`stage7Audit.ts`	A deterministic (non-LLM) internal-consistency audit — detailed in Section 7.3.
`formatIndex.ts`	Keyword-based retrieval over a fixed story-format reference corpus — detailed in Section 6.2.
`foundationDoc.ts`	Deterministic template compilation of confirmed canon into the fixed Story Foundation document schema.
`stripCatalogCodes.ts`	Boundary-enforcement utility — detailed in Section 7.5.
`authorType.ts`	A local, non-LLM heuristic classifier used only to bias internal defaults — detailed in Section 6.3.

2.2 Per-Project Engines: Configuration, Not Reimplementation

Each creative-development project has its own small directory (`characterEngine/` for the Character Bible, `worldEngine/` for the World Bible) that holds only: Story-Foundation ingestion logic, the project's own LLM tool schema, and any project-specific glue. Every file in both directories carries an identical scope declaration in its header comment, to the effect that canon-state tracking, conflict resolution, and document compilation belong exclusively to the shared engine and must not be reimplemented per project. This is demonstrated concretely, not just declared: the Character Bible project passes a distinct Firestore collection name (`CHARACTER_FACTS_COLLECTION`) into the exact same `canonStore.ts` functions used by the Story Foundation project, rather than maintaining a parallel state-machine implementation.

> **VERIFIED FINDING**
>
> The Character Bible engine (Project 2) is materially more complete than the World Bible engine (Project 3) at the time of this disclosure: Project 2 has its own conflict resolution, causal-chain validation, and document compiler; Project 3 currently has only ingestion and a turn schema, with canon-state tracking, guardrails, and conflict resolution for that project tracked as separate, not-yet-implemented work items.

3. Data Flow — The Turn-Processing Pipeline

Every conversational turn in every project follows the same pipeline shape. The following is traced end-to-end through the World Bible project's chat route, with the Story Foundation project's route (the most fully built of the three) cited for the additional steps it performs.

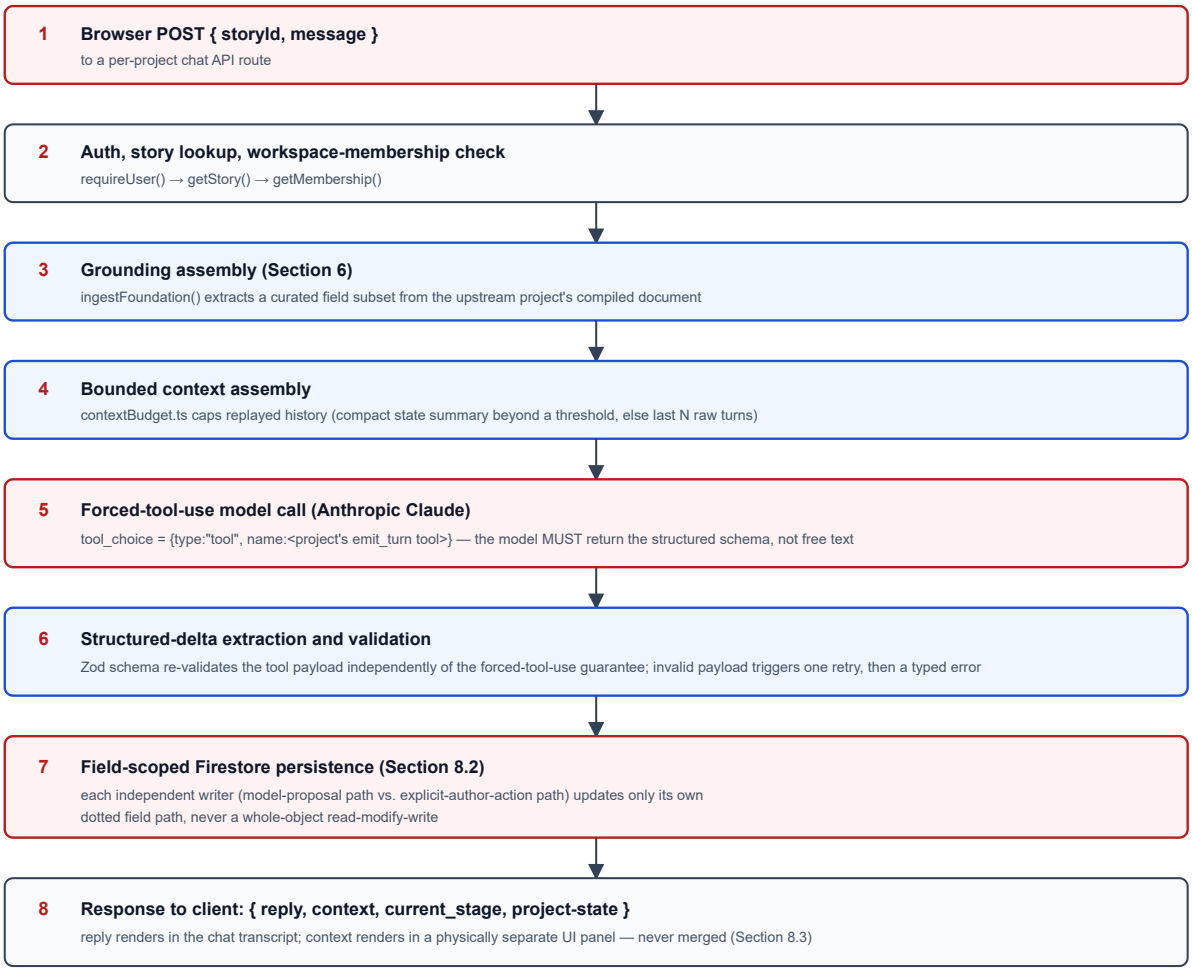


Figure 2 — The turn-processing pipeline, traced through the World Bible project's chat route and cross-checked against the Story Foundation project's route.

The forced-tool-use contract for the World Bible project, read directly from source, illustrates step 5 concretely:

```
export const EMIT_WORLD_TURN_TOOL: Anthropic.Tool = {
  name: "emit_world_turn",
  input_schema: {
    type: "object",
    properties: {
      reply:   { type: "string", description: "chat-facing reply... never narrate internal
bookkeeping" },
      context: { type: "string", description: "internal reasoning... shown separately, never
inside reply" },
      current_stage: { type: "number" },
```

```
      proposed_wcl:  { type: ["number","null"], enum: [1,2,3,4,null] },
    },
    required: ["reply","context","current_stage","proposed_wcl"],
  },
};
// called with tool_choice: { type: "tool", name: "emit_world_turn" } — forced, not "auto"
```

The Story Foundation project's route performs the same eight-step pipeline with additional steps interleaved: a local (non-LLM) author-type classification heuristic, a multi-stage-gate advancement loop capable of crossing more than one stage boundary per turn, a keyword-retrieval grounding injection scoped to a single stage, and a three-way conflict-resolution branch — all detailed in Sections 4.3, 6.2, and 6.3.

4. Canon Management Mechanism

4.1 The Canon State Machine

Every tracked narrative fact (a character trait, a world-building decision, a story-structure element) is represented as a `CanonElement` with exactly one of four states: `Exploring` (under discussion), `Working` (a provisional choice), `Confirmed` (the author has approved it as canon), or `Parked` (deferred — surfaced to the author-facing UI as "Deferred" in later projects). Confirmed elements can only move directly to Parked; any other change to a Confirmed element must route through the Conflict Resolution Protocol (Section 4.3) rather than being applied directly.

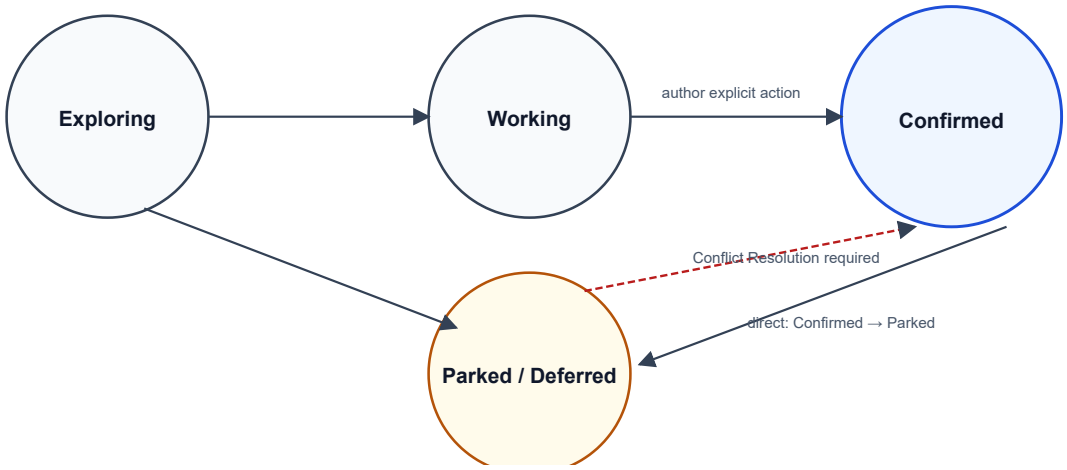


Figure 3 — The canon-element state machine. Any change to an already-Confirmed element other than a direct move to Parked must pass through the Conflict Resolution Protocol.

> **DESIGN PRINCIPLE, VERIFIED IN BOTH PRD AND CODE**
>
> State transitions are required to be explicit author actions — never inferred from conversational tone. This is stated directly in the World Bible product requirements ("State transitions must be explicit author actions (button/command), never inferred silently from conversational tone") and is demonstrably implemented, not just aspirational: the World Complexity Level mechanism (Section 8.1) gives the model a write path only to a "proposed" field; only an explicit UI action (a discrete PATCH request triggered by a button) can write the author-confirmed field.

4.2 Systemic Dependency Graph

Canon elements can declare a `depends_on` list referencing other element identifiers. A reverse-dependency lookup (`listDependents()`) is implemented as a single Firestore `array-contains` query, used to warn an author when a change to one element affects another that references it.

> **IMPLEMENTATION GAP, EXPLICITLY NOTED**
>
> The product requirements specify a full cascade — dependents of dependents, recursively — described as "an automatic Dependency Review prompt listing everything downstream." Only a single-level reverse lookup is currently implemented; a multi-level breadth-first cascade is documented internally as a known gap and was confirmed absent from the source tree, not merely undocumented.

4.3 Conflict Resolution Protocol

When a proposed change would alter an already-Confirmed element, the system detects the conflict before writing, constructs a system-level context message describing the contradiction, and requires the author to choose one of three resolutions: revert the new idea, revise the existing canon (triggering a downstream cascade review), or explicitly defer the new idea. This protocol is fully implemented, with real detection and resolution code, for the Story Foundation project. The Character Bible project implements a variant of the same protocol against the Story Foundation's canon rather than its own: because there is no deterministic way to algorithmically judge whether a new psychological detail contradicts an established character trait, the *detection* of a contradiction is model-declared (the model states that a conflict exists) while the *consequence* — gating further progress until the author resolves it — is enforced deterministically in code. The World Bible project does not yet have a code-level implementation of this protocol; its system prompt instructs the model to run the three-way choice conversationally, but no code enforces it.

5. AI/ML Implementation Details

5.1 Model and Forced Structured Output

All three implemented projects call the same underlying model (`claude-sonnet-5` , via the `@anthropic-ai/sdk` Node client) using an identical mechanism for guaranteeing machine-parseable output: the API call sets `tool_choice` to force the single project-specific "emit turn" tool, rather than allowing the model to choose between a free-text reply and a tool call. This displaces the reliability problem of "does the model remember to call the tool" onto the API layer itself. As defense in depth against occasional malformed structured output (observed and remediated in production — see Section 9.2), the tool payload is independently re-validated against a Zod schema after extraction, with one automatic retry before surfacing a typed validation error.

5.2 Per-Project System Prompts

Each project is driven by its own distinct, versioned system-prompt document (not a single shared prompt with project flags):

File	Persona
`sp01-sdos-systemprompt.md`	Expert Development Editor, Story Consultant, and Creative Writing Partner (Story Foundation)
`sp02-cdc-systemprompt.md`	Expert Character Development Consultant, Narrative Psychologist (Character Bible)
`sp03-wdc-systemprompt.md`	Expert World Development Consultant, Anthropologist/Sociologist, Speculative Worldbuilder (World Bible)

Each prompt independently defines: an adaptive depth/complexity framework, explicit scope-boundary rules directing the model to defer out-of-lane topics to the correct other project, and its own canon-state vocabulary section — reinforcing at the prompt layer the same state-machine concepts enforced independently in code (Section 4.1).

6. Retrieval and Grounding Methodology

6.1 Cross-Project Structured Field Grounding

When a later project needs to be grounded in an earlier project's output, the mechanism is **structured field extraction followed by direct text interpolation into the system prompt** — not injection of the full upstream document, and not a second model call to summarize it. A dedicated ingestion function fetches the latest compiled version of the upstream project's document, extracts a small, fixed, curated set of fields (for the World Bible project: working title, genre/tone, premise, and the Story-World section; for the Character Bible project, additionally a parsed list of principal cast members), and guards each extraction so that a missing or malformed section degrades to an explicit "incomplete" status rather than throwing an error. The extracted fields are interpolated into the system prompt at call time as a labeled grounding block, recomputed fresh from the database on every request:

```
system += `\n\n[Story Foundation grounding - computed by the app, trust this over
re-deriving it. Internal grounding only, never narrate this raw data to the author.]
Working Title: ${foundation.workingTitle || "(not set)"}
...`;
```

6.2 Keyword Retrieval Over the Story-Format Library

Separately, the Story Foundation project's format-diagnosis stage performs genuine information retrieval: a fixed, locally stored reference corpus of story formats is scored against the current conversation using term-frequency / inverse-document-frequency (TF-IDF) keyword matching, and the top-scoring candidates are injected as a distinct grounding block. This retrieval step is explicitly scoped to a single stage of the Story Foundation workflow and is not used elsewhere.

6.3 What Is Explicitly Not Present

> **VERIFIED FINDING — IMPORTANT FOR SCOPE OF DISCLOSURE**
>
> A full-repository search for embeddings, vector search, fine-tuning, or retrieval-augmented-generation infrastructure (search terms: embedding, vector, fine-tune, RAG, similarity search, and named vector-database products) returned no implementation matches. The one relevant hit is a code comment in the format-retrieval module explicitly disclaiming the approach: "Pure keyword/TF-IDF scoring, no embeddings, no external API." The system's entire "intelligence" layer is therefore: (a) prompt engineering via versioned, persona-specific system prompts, (b) forced structured tool output validated independently in code, and (c) deterministic, hand-written state machines and heuristics layered on top — not model training, fine-tuning, or learned/embedding-based retrieval of any kind. A separate, purely local heuristic (regex-based "author type" classification, used only to bias internal depth defaults and never shown to the author) is implemented without any additional model call, reinforcing that the system deliberately favors deterministic, auditable logic over additional ML inference wherever the product requirement allows it.

7. Consistency-Checking Processes

Two distinct mechanisms exist under the general heading of "guardrails," and they serve different purposes — worth distinguishing precisely for disclosure purposes.

7.1 Prompt-Level Scope Boundaries

Each system prompt contains its own explicit scope-boundary section instructing the model which topics to defer, and to which other project. This enforcement is entirely prompt-based; no code inspects a turn's content to detect or block an out-of-scope answer. This is a requirements-specified, prompt-implemented, not code-enforced mechanism.

7.2 Passive Heuristic Guardrail Logging

A separate, genuinely code-level mechanism regex-scans both the chat-facing reply and the internal-reasoning context of every turn for four specific failure signatures: an excessive-questioning pattern (more than three question marks in one reply, a "questionnaire dump"), leakage of internal stage/canon-state narration into the chat-facing text, verbatim leakage of system-prompt section headers, and leakage of internal schema/classification terminology. This mechanism is explicitly documented in its own source as passive: flagged turns are written to a dedicated Firestore subcollection and surfaced only in an internal debug panel for prompt-tuning review — the mechanism never blocks, alters, or delays the reply shown to the author.

7.3 Deterministic Stage Audit

The Story Foundation project runs a fully deterministic (non-LLM) internal-consistency audit before allowing the interview to advance out of its penultimate stage: word-overlap heuristics check whether the drafted Climax addresses the same terms as the Core Dramatic Question, and whether the Transformation Arc aligns with the stated Theme Statement, alongside a cross-check against a list of common structural mistakes. This runs as ordinary application code, not as an additional model call, and the interview cannot proceed to document generation until the author acknowledges the audit summary.

7.4 Causal-Chain Traceability Gate

The Character Bible project enforces a deterministic, code-level rule specific to psychological causality: a Confirmed core-flaw or dominant-fear fact is only permitted to remain in the Confirmed state if it declares a dependency on at least one already-Confirmed core-wound or false-belief fact for that character. This check is evaluated against both the current persisted state and any other facts being confirmed in the same turn, and is scoped to characters above a configured narrative-importance tier. This is a deterministic, app-level enforcement of a specific narrative-psychology causal model — not a general schema constraint, and not delegated to the model's judgment.

7.5 Retrieval-Metadata Boundary Enforcement

Internal catalog codes used for the story-format retrieval mechanism (Section 6.2) are tracked strictly as internal element metadata, are validated against the real format index (a hallucinated code is silently discarded rather than trusted), and are guaranteed never to reach an author-facing surface or an exported document: a dedicated utility recursively strips the metadata field from any compiled output and additionally scrubs any stray substring that matches the code's shape from free text. The same rule is enforced a second, independent time at the schema-validation layer, which rejects a turn where a catalog-code-shaped string appears in the wrong field. This is a layered (prompt instruction + document-compilation stripping + schema-level rejection) boundary enforcement for one specific class of internal-only data.

8. Distinguishing Technical Mechanisms

The following mechanisms are called out specifically because each solves a concrete, previously-observed technical failure mode with a specific, non-obvious implementation choice, rather than being an incidental consequence of an otherwise generic chat application.

8.1 Proposal / Confirmation Field Split

Rather than storing a single mutable value for an author-facing decision, the system stores the model's current proposal and the author's confirmed decision as two independently-addressable fields (e.g. `proposedWorldComplexityLevel` versus `worldComplexityLevel`). The model is only ever permitted to write the proposal field; only an explicit, discrete author action (a button-triggered API call) writes the confirmed field. This keeps "what the model most recently suggested" and "what the author has actually decided" as two structurally distinct pieces of state, and once a value has been explicitly confirmed, later model proposals update only the proposal field and are never automatically applied — the author remains the sole authority over the confirmed value for the remainder of the session.

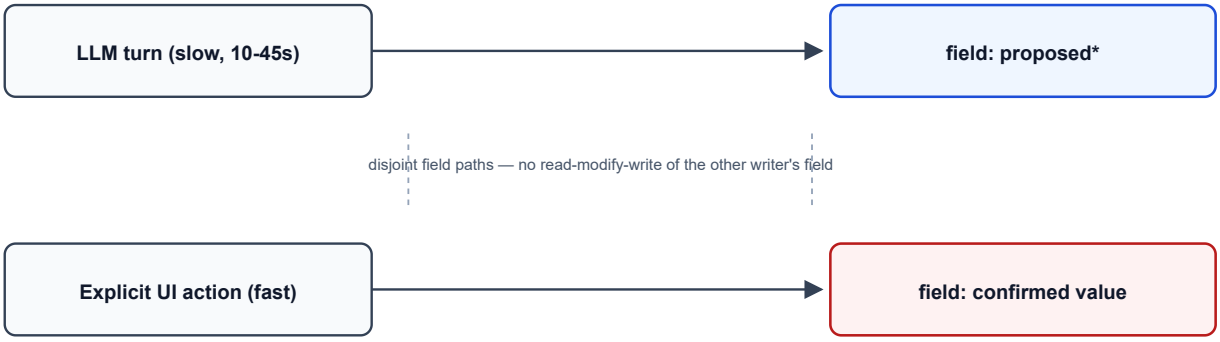


Figure 4 — Two independent writers, two disjoint fields: a slow LLM-driven proposal path and a fast explicit-UI confirmation path never contend for the same field.

8.2 Disjoint Dotted-Field-Path Concurrency Control

The mechanism above is implemented, at the database level, using Firestore's dotted-field-path partial-update syntax rather than a whole-document or whole-object read-modify-write:

```
export async function setP3ProposedLevel(storyId, level) {
  await storiesCollection().doc(storyId)
    .update({ "p3.proposedWorldComplexityLevel": level, updatedAt: ... });
}
export async function setP3ConfirmedLevel(storyId, level) {
  await storiesCollection().doc(storyId)
    .update({ "p3.worldComplexityLevel": level, updatedAt: ... });
}
```

> **VERIFIED FINDING — CONCRETE, PREVIOUSLY-SHIPPED BUG FIX**
>
> Source comments confirm this is not a hypothetical design precaution but a fix for an actual, previously-shipped race condition: an earlier whole-object read-modify-write implementation allowed a turn's stale in-memory snapshot of the shared state object (captured before a 10-45 second model call began) to silently

> overwrite an author's explicit confirmation if the author clicked "Confirm" while that model call was still in flight. Splitting each independent writer onto its own exclusive dotted field path makes the two write paths structurally non-interfering, eliminating the race entirely rather than mitigating it with locking, retries, or optimistic-concurrency version checks.

8.3 Dual-Channel Structured Output (Reply / Context)

Every project's forced tool schema requires two sibling string fields in the same response: a chat-facing `reply` and an internal-reasoning `context` field, explicitly documented as never to be blended. This guarantees, from a single model call, both a concise author-facing message and a separate internal-reasoning trace — avoiding two alternatives considered and rejected: relying on the model to opportunistically mix a free-text block with a tool-use block under a non-forced `tool_choice` setting (observed to be unreliable), or making two separate round-trip model calls per turn (rejected on cost/latency and product-requirement grounds). The two fields are kept separate all the way to the UI: the reply renders in the chat transcript; the context renders in a physically distinct panel component, verified end-to-end from schema to rendered output.

8.4 Configurable Shared State-Machine Engine

Rather than each creative-development project implementing its own canon-tracking persistence and transition logic, every project configures one shared, parameterized engine (Section 2.1) — demonstrated concretely by the Character Bible project passing a distinct Firestore collection name into the identical store functions used by the Story Foundation project.

8.5 Causal-Chain Gate on Confirmed Psychological State

Described in Section 7.4: a deterministic, code-enforced dependency requirement gating which psychological facts are permitted to remain in the Confirmed canon state, distinct from — and independent of — the general Confirmed/conflict-resolution mechanism.

8.6 Layered Retrieval-Metadata Boundary Enforcement

Described in Section 7.5: internal-only retrieval metadata is enforced out of author-facing and exported surfaces at three independent layers (prompt instruction, deterministic document-compilation stripping, and schema-level rejection), rather than relying on any single layer.

9. Technical Effects Achieved

- **Elimination of a specific, previously-observed race condition** between long-running LLM inference and fast, explicit human decisions, achieved structurally (disjoint field-level writes) rather than through locking, optimistic-concurrency retries, or eventual-consistency reconciliation.
- **Preservation of human authority over AI-suggested state**: once an author has explicitly confirmed a value, no subsequent model output can silently alter it, while the model's ongoing suggestions remain visible and available for the author to adopt at will.
- **Reliable machine-checkable output from a probabilistic text model**, achieved by combining API-level forced tool-use with independent schema re-validation, rather than relying on either mechanism alone — with a documented, previously-observed failure mode (truncated structured output under an insufficient token budget) that this defense-in-depth approach was built to catch.
- **Separation of audit/debug information from end-user-facing content** without a separate request or model call, by requiring both to be produced together in one structured response and routed to physically distinct UI surfaces.
- **Guaranteed structural conformance of generated documents**: the final compiled output documents are produced by deterministic template-fill over confirmed canon state, with the model used only for prose synthesis inside fixed sections — never as a single free-form generation of the whole document — so structural conformance to a fixed schema is guaranteed by construction rather than hoped for.
- **Reduced marginal engineering cost per additional creative-development stage**, because each new stage configures a shared, already-hardened state-management engine (persistence, transitions, conflict handling, structured-output extraction) rather than re-implementing it, evidenced by the measurable difference in maturity between the Character Bible and World Bible projects at time of writing.
- **Bounded, cost-controlled conversational context**: a context-budgeting mechanism prevents unbounded growth of the prompt sent on every turn as a session grows long, substituting a compact state summary for full transcript replay beyond a configured threshold.

10. Current Implementation Status and Known Limitations

For an accurate disclosure, the following distinctions between specified and implemented behavior are noted explicitly:

Mechanism	Story Foundation (P1)	Character Bible (P2)	World Bible (P3)
Canon state machine	Fully implemented	Fully implemented (shared engine)	Not yet implemented
Conflict resolution	Fully implemented	Implemented (model-declared detection, code-enforced consequence)	Prompt-only, no code enforcement
Dependency graph	Single-level reverse lookup only	—	Not yet implemented
Scope-boundary guardrails	Prompt-only	Prompt-only	Prompt-only
Two-field proposal/confirmation split	—	—	Implemented (World Complexity Level)

Additionally: multi-level (breadth-first) dependency cascading beyond one hop is specified in the product requirements but confirmed absent from the source tree. The Screenplay Architecture and Draft Writing projects (referred to in planning documents as later stages) have no corresponding implementation directories at the time of this disclosure — they exist only as product requirements and prompt drafts. Firebase Authentication is scaffolded and called from every route, but was noted in internal architecture documentation as not fully wired at an earlier point in development; current status was not independently re-verified beyond confirming the calling pattern is present throughout the code.

11. Glossary

Canon Element

A single tracked narrative fact, carrying a status, a value, a rationale, and a dependency list.

Canon Engine

The shared library implementing canon-element state, transitions, conflict resolution, structured-output extraction, and document compilation, consumed by every project.

Forced tool-use

An LLM API call mode in which the model is required to invoke a specific named function/tool, rather than being allowed to choose between free text and a tool call.

Grounding block

A labeled block of text, computed by the application and appended to the system prompt at call time, containing a curated set of upstream facts.

Project

One stage of the overall authoring workflow (Story Foundation, Character Bible, World Bible, and later stages), each with its own chat interface, system prompt, and (where implemented) canon tracking.

Structured delta

The machine-checkable payload a model returns each turn, describing its current view of tracked state, validated against a schema before being persisted.

TF-IDF

Term-frequency / inverse-document-frequency — a classical, non-learned keyword-weighting technique used for the story-format retrieval mechanism; explicitly not an embedding-based technique.

Appendix A — Source File Inventory (Primary References)

Path	Relevance
`ARCHITECTURE.md`	Living design-intent record; treated as intent, not as a verified changelog
`web/src/lib/canonEngine/*.ts`	Shared Canon Engine — Sections 2.1, 4, 7.3, 7.5
`web/src/lib/characterEngine/*.ts`	Character Bible per-project glue — Sections 2.2, 4.3, 7.4
`web/src/lib/worldEngine/*.ts`	World Bible per-project glue — Sections 2.2, 3, 8.1
`web/src/lib/turnGuardrails.ts`	Passive heuristic guardrail logging — Section 7.2
`web/src/app/api/chat/route.ts`	Story Foundation turn-processing route — Section 3
`web/src/app/api/character-chat/route.ts`	Character Bible turn-processing route — Section 3
`web/src/app/api/world-chat/route.ts, world-chat/wcl/route.ts`	World Bible turn-processing route and explicit-confirmation route — Sections 3, 8.1
`web/system-prompts/sp01-*.md, sp02-*.md, sp03-*.md`	Per-project system prompts — Section 5.2
`docs/superpowers/specs/2026-08-25-p3-world-complexity-level-design.md`	Design rationale for the proposal/confirmation split — Section 8.1
`project-docs/storimac-prds/*.md`	Product requirements referenced for specified-vs-implemented comparisons — Section 10

This document was compiled by direct inspection of the production source repository on 26 August 2026. It is a factual technical brief intended as raw material for IP assessment and disclosure drafting; it does not itself constitute legal claim language.